

MÁSTER

GESTIÓN Y ALMACENAMIENTO DE INFORMACIÓN NO ESTRUTURADA

TRABAJO PRÁCTICO:
BASES DE DATOS COLUMNARES: CASSANDRA



Versión 8.0

MÁSTER UNIVERSITARIO EN INGENIERÍA Y CIENCIA DE DATOS

Índice

Índice 2
Consideraciones iniciales: 3
Presentación y requisitos 3
Ejercicio 1 3
Apartado 1: Levantando la Infraestructura con Docker Compose 4
Apartado 2: Ejecutando CQL desde la consola 6
Apartado 3: Conexión al clúster con DBeaver 7
Ejercicio 2 10
Apartado 1: Diseño del cluster y la base de datos 11
Apartado 2: población de datos 11
Apartado 3. Resolución de Consultas 12
Apartado 4. Limitaciones del modelo y consultas restringidas 12
Apartado 5. Consultas no planificadas y Anti-patrones 12
Apartado 6. Alta Disponibilidad y Nivel de Consistencia 13
Documentación a entregar 13
Formato de subida 14
Enlaces de interés 14
Comentario final 14

Consideraciones iniciales:

En este documento se presenta el Trabajo Práctico (TP) del Módulo 4, "**Bases de datos columnares: Cassandra**", perteneciente a la asignatura *Gestión y Almacenamiento de Información No Estructurada* del Máster Universitario en Ingeniería y Ciencia de Datos de la UNED.

Este trabajo tiene carácter **estrictamente individual**. A continuación, se detallan algunas pautas generales para su realización:

- **Entorno de trabajo:** Se puede emplear cualquier sistema operativo (Windows, macOS o distribuciones de Linux).
- **Lectura atenta:** Se recomienda leer detenidamente todas las indicaciones de este enunciado para evitar errores, tanto en la implementación técnica como en la elaboración de los materiales requeridos para la entrega final.
- **Soporte y dudas:** Como es habitual, el canal oficial y más efectivo para plantear cualquier duda, solicitar ayuda o compartir sugerencias son los foros de la asignatura.

Presentación y requisitos

En este documento se proponen 2 ejercicios prácticos para que el estudiante se familiarice con todos los aspectos del despliegue, la arquitectura y la utilización de bases de datos NoSQL columnares y, en concreto, de la tecnología Apache Cassandra.

En el **primer ejercicio** el objetivo será puramente de **infraestructura y operación**. El alumno tendrá que desplegar una arquitectura distribuida real levantando un clúster de 3 nodos en su propia máquina utilizando **Docker Compose**. Posteriormente, configurará la conexión a dicha infraestructura utilizando el cliente gráfico **DBeaver** para poder interactuar con ella de forma cómoda.

En el **segundo ejercicio**, se explorará a fondo el **modelado y diseño** de bases de datos utilizando Cassandra a través de un caso de uso empresarial (gestión de flotas logísticas). Lo más importante es que el alumno interiorice el diseño guiado por consultas (*Query-Driven Modeling*), comprenda la necesidad de desnormalizar la información frente al modelo relacional tradicional, evite anti-patrones de rendimiento y ponga a prueba la tolerancia a fallos del clúster levantado en el primer ejercicio.

Ejercicio 1

En el mundo real, los sistemas modernos que manejan grandes volúmenes de datos (Big Data) o que requieren estar siempre disponibles (Alta Disponibilidad) no se ejecutan en un solo servidor, sino en redes de múltiples servidores trabajando en equipo.

El objetivo de esta práctica es que aprendáis a desplegar, configurar y operar **Apache Cassandra**, una de las bases de datos NoSQL distribuidas más potentes y utilizadas en la industria tecnológica.

Para que el aprendizaje sea completo, este primer ejercicio está dividido en tres apartados:

- **Apartado 1: Levantando la infraestructura con Docker Compose.** Utilizaremos Docker para "levantar" un clúster de 3 servidores (nodos) de Cassandra dentro de vuestro propio ordenador.
- **Apartado 2: Ejecutando CQL desde la consola.** Interactuaremos directamente con el clúster a través de su línea de comandos nativa (cqlsh) para entender cómo se procesan las sentencias a bajo nivel.
- **Apartado 3: Conexión al clúster con DBeaver.** Utilizaremos un cliente gráfico para conectarnos a nuestra nueva infraestructura y gestionarla de forma visual e intuitiva.

Apartado 1: Levantando la Infraestructura con Docker Compose

En esta primera parte, vamos a simular un centro de datos real utilizando **Docker** para crear una infraestructura distribuida de **3 nodos** (servidores virtuales) que se comunicarán entre sí formando un anillo. Para lograr esto de forma orquestada, utilizaremos el archivo `docker-compose.yml` que se os ha proporcionado.

⚠️ PRE-REQUISITO FUNDAMENTAL: > Antes de abrir la terminal o intentar ejecutar cualquier comando, es obligatorio que el motor (daemon) de Docker esté arrancado en tu sistema, ya uses Windows, macOS o Linux.

- Si usas Windows o Mac, abre la aplicación **Docker Desktop** y espera a que el icono (la ballena) indique que el sistema está corriendo ("*Docker Desktop is running*").
- Si intentas ejecutar los comandos sin arrancar Docker primero, la terminal te devolverá un error de conexión.

Entendiendo el archivo *docker-compose.yml*

Antes de ejecutar ningún comando, es crucial entender qué le estamos pidiendo a Docker que construya. Si abrís el archivo, veréis que está dividido en varias secciones clave:

- **Servicios (Los Nodos):** Definimos tres contenedores llamados `cassandra-node1`, `cassandra-node2` y `cassandra-node3`. Todos utilizan la imagen oficial más reciente de Cassandra (`image: cassandra:latest`).
- **Mapeo de Puertos (`ports`):** Cassandra se comunica por defecto a través del puerto `9042`. Como no podemos tener tres máquinas virtuales usando el mismo puerto en vuestro ordenador físico, los hemos mapeado así:
  - Nodo 1 responde en el **9042** (`9042:9042`).
  - Nodo 2 responde en el **9043** (`9043:9042`).
  - Nodo 3 responde en el **9044** (`9044:9042`).
- Variables de Entorno (**`environment`**):
  - `CASSANDRA_CLUSTER_NAME`: Todos los nodos tienen el mismo nombre (**`ClusterClaseDB`**) para saber que pertenecen a la misma "familia".
  - `CASSANDRA_SEEDS`: Indicamos que el `cassandra-node1` es el "nodo semilla". Es decir, es el primer punto de contacto al que los nodos 2 y 3 deben "llamar" para unirse al anillo.
  - **La "Dieta de RAM":** Cassandra es un sistema basado en Java que consume mucha memoria. Hemos limitado estrictamente la memoria de cada nodo a 512 MB (`JVM_OPTS=-Xms512M -Xmx512M`) para evitar que vuestros ordenadores colapsen al arrancar los tres a la vez.
- **Dependencias de Arranque (`depends_on`):** Le indicamos a Docker que los nodos 2 y 3 no deben intentar arrancar hasta que el nodo 1 esté 100% operativo (`service_healthy`). Esto evita cuellos de botella en la CPU de vuestro equipo.
- **Volúmenes (`volumes`):** Los contenedores son efímeros (si se borran, se pierde todo). Para evitar esto, creamos "discos duros virtuales" (`node1_data`, etc.) conectados a la ruta interna `/var/lib/cassandra`. Así, vuestros datos sobrevivirán a los reinicios.

Comandos de Operación Básica: Levantando el Clúster

Con Docker ya encendido en tu ordenador, abre tu terminal de comandos, navega hasta la carpeta donde has guardado el archivo `docker-compose.yml` y ejecuta los siguientes pasos:

Paso 1: Arrancar la infraestructura

```
docker-compose up -d
```

El parámetro -d (detached) indica que los contenedores se ejecutarán en segundo plano, dejándote la terminal libre para seguir trabajando.

Paso 2: Comprobar el estado de Docker

Para verificar que los tres contenedores están corriendo y no se han apagado por falta de recursos, ejecuta:

```
docker ps
```

Deberías ver una lista con los tres contenedores y su estado (*Up*).

Paso 3: Comprobar el Anillo (Ring) de Cassandra

Aunque Docker indique que los contenedores están encendidos, el motor de base de datos tarda entre 1 y 2 minutos en arrancar internamente y formar el clúster distribuido. Para comprobar el estado real de la red de Cassandra, ejecutamos la herramienta `nodetool` dentro del nodo 1:

```
docker exec -it cassandra-node1 nodetool status
```

- Si ves letras **UJ** (*Up / Joining*), los nodos aún se están conectando. Espera un minuto y vuelve a lanzar el comando.
- Cuando veas tres filas con las letras **UN** (*Up / Normal*), ¡tu clúster estará 100% estabilizado y listo para trabajar!

Paso 4: Comprobar la creación de Volúmenes

Para verificar que Docker ha reservado correctamente el espacio físico en tu disco duro para guardar los datos de forma permanente:

```
docker volume ls
```

Resolución de Problemas (Troubleshooting)

En sistemas distribuidos, es habitual que algún nodo falle por picos de consumo (especialmente de memoria RAM durante el arranque). Si al hacer `docker ps` o `nodetool status` descubres que falta un nodo o aparece como **DN** (*Down / Normal*), utiliza estas herramientas:

Ver los logs (Registro de errores) de un nodo:

Si, por ejemplo, el nodo 2 ha fallado, puedes leer sus últimos 50 mensajes de sistema para entender el porqué:

```
docker logs cassandra-node2 --tail 50
```

Arrancar un nodo individualmente:

Si un nodo se ha apagado repentinamente, puedes forzar su encendido manual sin afectar al resto del clúster:

```
docker start cassandra-node2
```

Tras arrancarlo, espera un par de minutos y vuelve a ejecutar *`nodetool status`* para verificar que se reincorpora al anillo de datos.

Ciclo de Vida del Clúster

No vais a dejar la base de datos encendida consumiendo recursos para siempre en vuestro PC personal. Aquí tenéis el "panel de mandos" para gestionar el ciclo de vida de vuestra infraestructura desde la terminal:

Pausar el clúster (Hibernación)

Detiene temporalmente los procesos sin apagar el contenedor. Útil si necesitas recursos de tu PC por un momento.

```
docker-compose pause
docker-compose unpause    # Para reanudar
```

Apagar el clúster (sin destruir)

Apaga los contenedores ordenadamente. La próxima vez que los enciendas, arrancarán más rápido y mantendrán su configuración.

```
docker-compose stop
docker-compose start      # Para volver a encenderlos
```

Destruir la infraestructura (manteniendo los datos)

Elimina los contenedores y las redes virtuales, pero **NO** borra los volúmenes. Al hacer esto, el entorno queda limpio, pero tus tablas y datos insertados sobrevivirán para la próxima vez que hagas `docker-compose up -d`.

```
docker-compose down
```

Borrado Nuclear (Destruir TODO)

Si tu clúster se ha corrompido, has cometido un error irreversible o has terminado la práctica por completo, este comando elimina contenedores, redes y **también borra los discos duros virtuales (volúmenes)**. Empezarás desde cero absoluto y los datos se perderán para siempre.

```
docker-compose down -v
```

Apartado 2: Ejecutando CQL desde la consola

Para comprobar que el despliegue se ha realizado correctamente y que el clúster es totalmente funcional, realizaremos una primera conexión directa a la base de datos. Para ello, utilizaremos **CQL (Cassandra Query Language)**, el lenguaje nativo de Cassandra, que es muy similar al SQL tradicional pero adaptado a sistemas distribuidos.

Una de las características principales de Apache Cassandra es su arquitectura descentralizada (*masterless* o *peer-to-peer*). Esto significa que no hay un nodo "maestro"; todos son iguales. Por lo tanto, podemos conectarnos a cualquiera de los tres nodos de nuestro clúster (`cassandra-node1`, `node2` o `node3`) para ejecutar nuestras operaciones y el sistema se encargará internamente de coordinar y replicar la información.

Para esta prueba, accederemos a la consola interactiva a través del Nodo 1 ejecutando el siguiente comando en nuestra terminal:

```
docker exec -it cassandra-node1 cqlsh
```

Una vez que el cursor cambie a `cqlsh>`, indicando que estamos dentro de la base de datos, ejecuta el siguiente bloque de sentencias paso a paso:

```
-- 1. Crear un Keyspace (equivalente a una base de datos)
-- El factor de replicación 3 garantiza que cada dato se copie
-- en nuestros 3 nodos.
CREATE KEYSPACE laboratorio
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 3};
```

```
-- 2. Seleccionar el Keyspace activo
USE laboratorio;

-- 3. Crear una tabla sencilla
-- En Cassandra, la clave primaria es crucial para decidir dónde
-- se guarda físicamente el dato.
CREATE TABLE alumnos (
    matricula TEXT PRIMARY KEY,
    nombre TEXT,
    curso TEXT
);

-- 4. Insertar datos de prueba
INSERT INTO alumnos (matricula, nombre, curso)
VALUES ('A001', 'Ada Lovelace', 'Bases de Datos Avanzadas');

INSERT INTO alumnos (matricula, nombre, curso)
VALUES ('A002', 'Alan Turing', 'Sistemas Distribuidos');

-- 5. Consulta general: Recuperar todos los registros insertados
SELECT * FROM alumnos;

-- 6. Consulta filtrada: Recuperar un único registro por su clave primaria
SELECT * FROM alumnos WHERE matricula = 'A001';

-- 7. Desconectar y salir de la consola de Cassandra
exit
```

Si las consultas devuelven los datos esperados, ¡enhorabuena! Tu infraestructura distribuida está operativa. Ahora estamos listos para conectar herramientas gráficas externas.

Apartado 3: Conexión al clúster con DBeaver

Con nuestro centro de datos virtual (el clúster de Docker) ya funcionando en segundo plano, el siguiente paso es poder acceder a los nodos mediante un cliente gráfico. Para ello, en lugar de usar la terminal, vamos a utilizar **DBeaver**, una de las herramientas gráficas de gestión de bases de datos más populares del mercado.

El reto de DBeaver y Cassandra

DBeaver es una herramienta fantástica porque es "universal"; te permite conectarte a MySQL, PostgreSQL, Oracle y muchos otros sistemas de gestión de bases de datos desde un mismo programa. Sin embargo, tiene una particularidad comercial con Apache Cassandra: **su versión gratuita (Community) tiene bloqueado el controlador oficial preconfigurado de Cassandra**, reservándolo solo para los usuarios que pagan la licencia PRO.

¡Pero no hay problema! En el mundo del software libre siempre hay alternativas. En lugar de pagar una licencia, vamos a enseñarle a DBeaver cómo comunicarse con Cassandra configurando una **Conexión Genérica JDBC** totalmente gratuita y de código abierto.

¿Qué es una conexión JDBC y qué es un archivo `.jar`?

Para que DBeaver (desarrollado en Java) pueda comunicarse con Cassandra, necesitamos establecer un puente de software. Aquí es donde entran en juego dos conceptos fundamentales en la arquitectura de aplicaciones:

- **JDBC (Java Database Connectivity):** Es una **API** (Interfaz de Programación de Aplicaciones) estándar del ecosistema Java. Define un conjunto de reglas, clases e interfaces genéricas que permiten a los clientes SQL conectarse a cualquier motor de base de datos. Gracias a esta API,

DBeaver no necesita entender cómo funciona internamente Cassandra, Oracle o PostgreSQL; simplemente envía las peticiones utilizando los métodos estándar de JDBC.

- **El archivo `.jar` (Java ARchive):** Es el paquete que contiene el **Driver** (controlador) específico. Técnicamente, un `.jar` es un archivo comprimido que agrupa las clases de Java compiladas que *implementan* la API de JDBC para un sistema concreto. En nuestro caso, este archivo contiene la lógica de red necesaria para traducir las llamadas estándar de DBeaver a peticiones **CQL** (Cassandra Query Language) y manejar el protocolo binario subyacente que espera nuestro clúster.

La solución Open Source de ING Bank

Existen muchos controladores `.jar` en internet para Cassandra, pero configurar algunos de ellos es un muy complicado porque requieren descargar decenas de archivos secundarios de red por separado (dependencias).

Para esta práctica, utilizaremos el controlador de código abierto mantenido por el equipo de ingeniería del **Banco ING (ING Bank)**. Vamos a descargar la versión *Bundle* (también conocida como "Fat JAR") porque ya lleva todas las piezas y dependencias de red empaquetadas en un único archivo.

Instrucciones de Descarga:

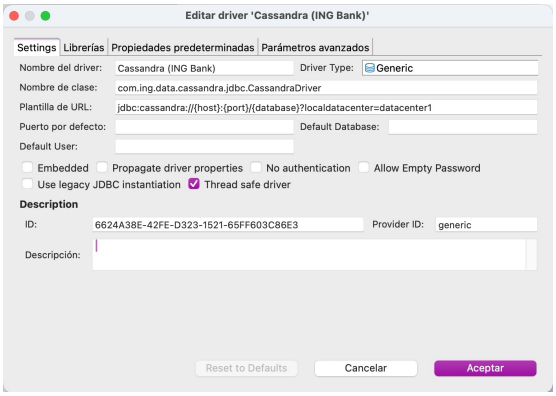
- Abre tu navegador y dirígete al repositorio oficial de lanzamientos en GitHub: https://github.com/ing-bank/cassandra-jdbc-wrapper/releases
- Busca el lanzamiento que tiene la etiqueta verde de "Latest" (actualmente la versión `4.16.3`).
- Al final de esa sección, despliega el menú llamado "Assets".
- Descarga ÚNICAMENTE el archivo que termina en **`-bundle.jar`** (el nombre exacto será `cassandra-jdbc-wrapper-4.16.3-bundle.jar`). Pesa alrededor de 30 MB.

Guarda ese archivo en una carpeta que tengas localizada (por ejemplo, en la misma donde guardaste el `docker-compose.yml`), ya que en el siguiente apartado se lo proporcionaremos a DBeaver como "cerebro" para conectarnos a nuestros nodos.

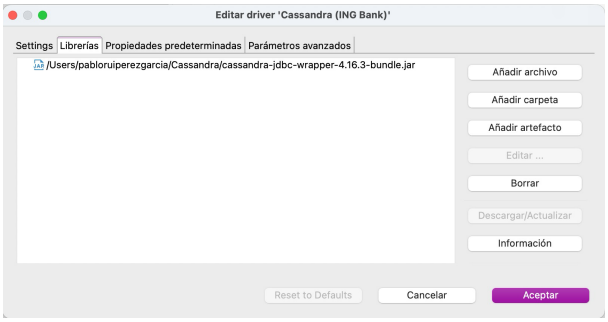
Configuración del Driver (El Motor)

Para que DBeaver reconozca el nuevo "adaptador" que hemos descargado, debemos registrarlo en su administrador de controladores:

- **Abrir el Administrador**: En el menú superior, ve a **Base de datos > Gestor de Driver**.
- **Crear Nuevo Driver**: Haz clic en el botón Nuevo.
- Ajustes Principales (Settings):
  - **Nombre del controlador**: `Cassandra (ING Bank)`
  - **Driver Type**: `Generic`
  - **Nombre de clase**: `com.ing.data.cassandra.jdbc.CassandraDriver` (Es la clase principal dentro del `.jar` que implementa la API).
  - **Plantilla URL**: `jdbc:cassandra://{host}:{port}/{database}?localdatacenter=datacenter1`



- **Cargar la Librería (Libraries):**
  - Cambia a la pestaña **Librerías**.
  - Haz clic en **Añadir Archivo**.
  - Busca y selecciona el archivo `cassandra-jdbc-wrapper-4.16.3-bundle.jar` que descargamos anteriormente.

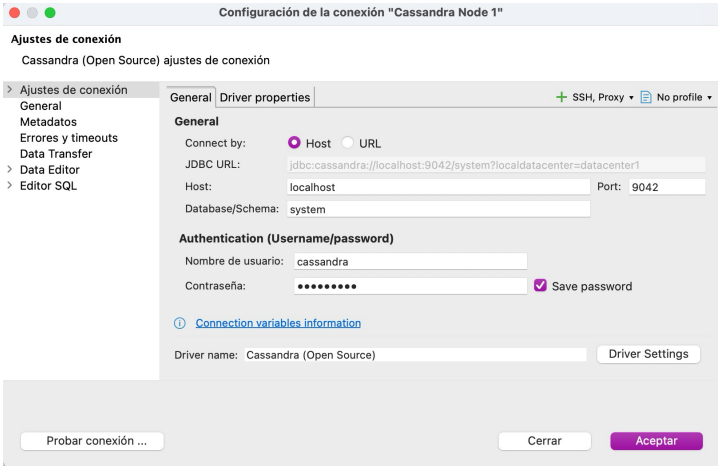


- **Finalizar:** Haz clic en **Aceptar**. Ahora DBeaver ya sabe cómo "hablar" con Cassandra.

Creación de la Conexión

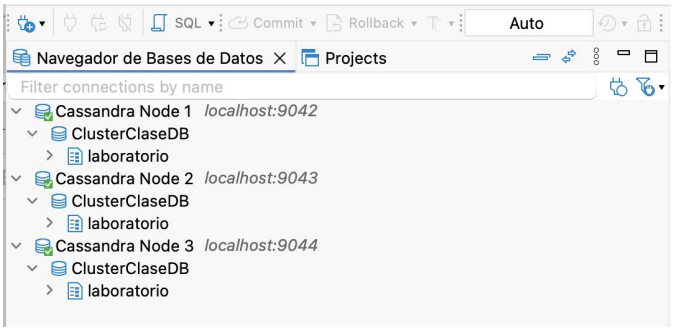
Ahora que el driver está instalado, vamos a crear la conexión específica a nuestro primer nodo de Docker:

- **Nueva Conexión:** Haz clic en el icono del enchufe con el símbolo "+" (**Nueva conexión**).
- **Seleccionar Driver:** Busca y selecciona el driver que acabamos de crear: `Cassandra (ING Bank)`.
- **Configuración de Red:**
  - **Host:** `localhost`
  - **Puerto:** `9042` (para el Nodo 1 del cluster)
  - **Database/Schema:** `system` (Es el Keyspace interno de Cassandra necesario para la conexión inicial).
- **Autenticación:**
  - **Usuario**: cassandra
  - **Contraseña**: cassandra
- **Prueba de Fuego:** Haz clic en **Probar conexión...**. Si todo es correcto, verás un mensaje de éxito indicando la versión de Cassandra (5.0.x).



Repite este proceso dos veces más para crear las conexiones al **Nodo 2** (Puerto `9043`) y **Nodo 3** (Puerto `9044`). Al finalizar, deberías tener tres conexiones activas en tu panel de "Navegador de bases de datos", permitiéndote saltar de un nodo a otro para observar la replicación de datos en tiempo real.

El resultado despúes de renombrar las conexiones y abriendo la conexión con los tres nodos, se muestra en la siguiente imagen donde se ve el Keyspace *laboratorio* creado en la consola anteriormente.



Ejercicio 2

Una empresa de logística y transporte está desarrollando una nueva aplicación que permite a los gestores de flota conectarse y recibir información en tiempo real sobre el estado y posibles incidencias de sus rutas comerciales.

Se desea almacenar la siguiente información:

- **idConductor**: identificador único (texto) de cada conductor.
- **idRuta**: identificador entero de cada ruta planificada. Es el mismo para todos los conductores asignados a esa misma ruta (por ejemplo, rutas largas que requieren turnos de relevo).
- **almacenOrigen**: nombre de la instalación o almacén de salida (texto).
- **almacenDestino**: nombre de la instalación o almacén de llegada (texto).
- **fechaRuta**: fecha de inicio de la ruta (tipo date).
- **horasEstimadas**: duración planificada de la ruta en horas (entero).
- **incidencias**: conjunto de eventos relacionados con la ruta (por ejemplo, averías, atascos, controles de peso...). Cada incidencia consta de una clave (tipo de incidencia) y un valor asociado, y puede actualizarse durante el transcurso de la ruta.

Las consultas más habituales que se desea realizar son:

1. Dado un **idConductor**, obtener el almacén de origen, el almacén de destino, la fecha de inicio y las horas estimadas de **todas sus rutas, ordenadas por fechaRuta** en orden descendente (es decir, primero las más recientes).
2. Dado un **idRuta**, obtener todos los conductores (idConductor) que participaron en ella junto con sus **incidencias, ordenados por idConductor** de forma ascendente.

Consideraciones adicionales:

- Tanto **idConductor** como **idRuta** ofrecen una buena distribución de los datos entre nodos.
- La combinación (**idConductor, idRuta**) debe ser única (un conductor no puede repetir el mismo identificador de ruta).

Apartado 1: Diseño del cluster y la base de datos

El primer paso en cualquier proyecto con Apache Cassandra no es pensar en las entidades y sus relaciones como haríamos en el modelo relacional, sino en las preguntas específicas que el sistema debe responder. Aplicando el principio de diseño guiado por consultas (*Query-Driven Modeling*), tu tarea es configurar el entorno distribuido inicial y diseñar un modelo de datos desnormalizado que garantice lecturas eficientes (O(1)) para los requisitos de la aplicación de logística.

- Crea un *keyspace* llamado logistica_`flotas` asumiendo que estamos en un clúster de **3 nodos** en producción. Utiliza la estrategia de replicación adecuada y configura un factor de replicación que asegure una copia en cada nodo.
- Basándote en las dos consultas habituales descritas en el enunciado, crea las tablas necesarias empleando tipos de datos adecuados (como colecciones para las incidencias). **Nota obligatoria:** Añade un comentario en tu código justificando técnica y funcionalmente tu elección de la Clave de Partición (*Partition Key*) y la Clave de Agrupamiento (*Clustering Key*) para cada tabla.

Apartado 2: población de datos

En el esquema creado en el apartado anterior, escribe el código CQL necesario para añadir la siguiente información empleando los correspondientes `INSERT`.

¡Atención a este cambio de paradigma! Si vienes del mundo de las bases de datos relacionales (SQL), te sorprenderá enormemente la necesidad de **desnormalizar** el modelo. En Cassandra, la **redundancia de datos** no es un error, sino una obligación. Está completamente justificado (y es necesario) insertar exactamente la misma información del viaje en varias tablas distintas. Este es el precio que pagamos en el almacenamiento para garantizar que las lecturas posteriores sean tremendamente rápidas y no requieran *JOINs*. Asegúrate de que todas tus tablas reciben los datos necesarios para funcionar.

1. Marcos87 conduce de **Madrid a París** el **2024-04-15**, con idRuta = **101**, duración de 5 horas. Incidencia: 'Aduana' : 'Retraso de 2 horas en frontera'
2. CarlosR participa en la misma ruta (idRuta = **101**) pero parte desde **Zaragoza**. Incidencia: 'Aduana' : 'Retraso de 2 horas en frontera'
3. EvaM33 conduce de **Valencia a Lisboa** el **2024-05-10**, con idRuta = **102**, duración de 8 horas. Incidencia: 'Avería' : 'Pinchazo en rueda delantera'
4. Marcos87 realiza otra ruta de **Barcelona a Milán** el **2024-06-01**, con idRuta = **103**, duración de 12 horas. Incidencia: 'Descanso' : 'Parada obligatoria de 45 mins'
5. Noel99 participa en la ruta idRuta = **103**, desde **Girona**. Incidencia: 'Descanso' : 'Parada obligatoria de 45 mins'

Apartado 3. Resolución de Consultas

Escribe y ejecuta el código CQL necesario para dar respuesta exacta a las siguientes peticiones, utilizando las tablas creadas:

1. Listar el origen, destino y fecha de todas las rutas en las que ha trabajado el conductor '**Marcos87**'.
2. Mostrar el identificador de todos los conductores y sus incidencias que participaron en la ruta **101**.
3. Obtener el origen, destino y duración de los viajes de **'Marcos87'** realizados a partir del **1 de mayo de 2024** (inclusive).
4. Consultar únicamente las incidencias que reportó **'CarlosR'** durante su participación en la ruta **101**.

Apartado 4. Limitaciones del modelo y consultas restringidas

Apache Cassandra prioriza la disponibilidad y la velocidad de lectura (latencia O(1)), lo que implica sacrificar cierta flexibilidad en las consultas. A diferencia de las bases de datos relacionales, Cassandra prohíbe operaciones que degraden el rendimiento del clúster.

A continuación, se plantean tres requerimientos. Escribe la sentencia CQL lógica que intentarías usar si estuvieras en SQL, ejecútala en Cassandra, y anota el **error exacto** que devuelve el sistema. Finalmente, **justifica el porqué de este error** basándote en la arquitectura y el modelo físico de almacenamiento de Cassandra.

1. **Ordenación global:** Se desea obtener un listado de **todas las rutas** del sistema, ordenadas por `fechaRuta` de forma descendente, consultando la tabla `rutas_por_conductor` (sin especificar ningún conductor).
2. **Búsqueda por rango en Clave de Partición:** Se necesita listar a todos los conductores que participaron en viajes cuyo identificador sea **mayor que 101**, consultando la tabla `conductores_por_ruta`.
3. **Actualización de Claves:** Por un error administrativo, la ruta de 'Marcos87' a París no fue el '2024-04-15', sino el '2024-04-16'. Intenta ejecutar una sentencia `UPDATE` sobre la tabla `rutas_por_conductor` para modificar esa `fechaRuta`.

Apartado 5. Consultas no planificadas y Anti-patrones

El departamento de operaciones solicita de urgencia un nuevo informe: obtener los identificadores de todas las rutas y sus conductores que inician su viaje en una fecha concreta (por ejemplo, el '2024-04-15').

Como la fecha (`fechaRuta`) no es la Clave de Partición en ninguna de nuestras tablas, **no es posible realizar esta consulta directamente**. Al no proporcionarle esta clave en el `WHERE`, Cassandra no puede aplicar su función matemática (el *hash*) y, por tanto, no tiene forma de saber en qué nodo del clúster se encuentran los datos, bloqueando la consulta por defecto.

Para resolver esta limitación, propón las siguientes tres alternativas:

- **Solución A (Forzar consulta):** Escribe el `SELECT` sobre las tablas existentes forzando la ejecución con `ALLOW FILTERING`. **Explica detalladamente** qué ocurre a nivel de arquitectura en el clúster al ejecutar esto y por qué es una práctica desastrosa en producción.
- **Solución B (Índices Secundarios):** Propón la creación de un índice secundario para la fecha y el `SELECT` resultante. **Explica** por qué esta herramienta oficial suele ser una mala práctica en Cassandra (*pista: investiga el patrón scatter-gather*).

- **Solución C (La vía "Cassandra"):** Propón la modificación óptima del modelo. Escribe el `CREATE TABLE` de una nueva tabla optimizada para esta búsqueda, inserta el registro de "Marcos87" en ella, y escribe el `SELECT` final. **Explica** por qué esta es la única solución escalable.

Apartado 6. Alta Disponibilidad y Nivel de Consistencia

En este ejercicio, se ha desplegado un clúster de **3 nodos**. La información está replicada en estos nodos siguiendo la estrategia y el factor de replicación (RF=3) que definimos al crear el KEYSPACE. En sistemas distribuidos, la disponibilidad depende de cuántas copias existan y de cuántos nodos deben confirmar una operación para considerarla válida. Comprobaremos cómo se comporta el clúster ante fallos reales de infraestructura ajustando el nivel de consistencia.

1. En Cassandra existen diversos niveles de consistencia (como ONE, TWO, THREE, QUORUM, ALL, etc.) que determinan cuántas réplicas deben responder. Configura el nivel de tu sesión actual a **QUORUM**.
2. Estando conectado a uno de los nodos (por ejemplo, el Nodo 1), simula un fallo en el sistema **apagando otro de los nodos** del clúster (el Nodo 2 o el Nodo 3). Con el nodo apagado, ejecuta la Consulta 1 del apartado anterior. **¿Qué ocurre y por qué?** Justifica tu respuesta basándote en la fórmula matemática del Quorum.
3. Simula un **segundo fallo** apagando otro nodo adicional (de modo que solo quede un nodo activo en todo el clúster). Vuelve a ejecutar la consulta. **¿Qué ocurre ahora y por qué?**

Documentación a entregar

Se deberá elaborar un documento principal en formato **PDF** comentando el trabajo realizado, acompañado de un archivo de texto con el código fuente.

Este documento PDF debe incluir secciones relativas a cada uno de los ejercicios que componen la práctica:

- Sobre el Ejercicio 1 (Infraestructura):

  Se incluirá una breve explicación de cómo ha sido el proceso de instalación, el despliegue del clúster local con Docker y las pruebas de conexión, comentando si ha habido alguna incidencia y cómo se ha resuelto.

  Se deben incluir pantallazos que evidencien el éxito del despliegue:

  - Captura donde se vean los **3 nodos (contenedores) en ejecución** (ya sea mostrando la salida de la terminal o desde la interfaz de Docker Desktop).
  - Captura interactuando con el clúster a través de la consola nativa (`cqlsh`).
  - Captura mostrando la conexión establecida correctamente desde el cliente gráfico **DBeaver**.

- Sobre el Ejercicio 2 (Modelado y Consultas):

  El documento debe estructurarse siguiendo fielmente los apartados del ejercicio. Para cada uno de ellos se debe incluir:

  - El **código CQL** desarrollado para resolver lo que se pide.
  - **Pantallazos** o capturas de pantalla que muestren la ejecución del código CQL y el resultado devuelto por la base de datos.
  - Las **explicaciones y justificaciones teóricas** solicitadas en el enunciado. Esto es especialmente crítico en la justificación de las claves (*Partition* y *Clustering Key*) del

Apartado 1, la justificación de los errores por limitación del modelo (Apartado 4), el análisis de los anti-patrones (Apartado 5) y las conclusiones sobre el Quorum (Apartado 6).

- **Archivo de código** (Texto plano):

  Junto al PDF, se entregará un documento de texto plano (extensión `.txt` o `.cql`) con todo el código CQL consolidado que se ha desarrollado en el Ejercicio 2. A este código se le deben añadir comentarios (usando `--`) indicando, al menos, a qué apartado corresponde cada bloque de sentencias.

Conclusiones: El documento PDF incluirá un apartado final de "Conclusiones". Se valorarán positivamente todos los comentarios sobre las impresiones del estudiante acerca del cambio de paradigma que supone este tipo de bases de datos, el aprendizaje obtenido y las dificultades encontradas a lo largo del trabajo práctico.

Formato de subida

Una vez finalizados todos los ejercicios, escrito el documento PDF y generado el fichero de texto con el código, el alumno debe comprimir todo en un único archivo `.zip` o `.rar` y subirlo a la plataforma virtual del curso. El nombre del fichero tiene que tener la siguiente estructura:

- `Cassandra2026_apellido1_apellido2_nombre.zip`
- `Cassandra2026_apellido1_apellido2_nombre.rar`

Enlaces de interés

- "*Cassandra: The Definitive Guide, (Revised) Third Edition*" by Eben Hewitt and Jeff Carpenter, O'Reilly Media, Inc. January 2022 (acceso libre con cuenta de la UNED)

  https://www.oreilly.com/library/view/cassandra-the-definitive/9781492097136/
- Sitio oficial de Cassandra: https://cassandra.apache.org/
  - Documentación de Cassandra: https://cassandra.apache.org/doc/latest/
  - Cassandra Ecosystem: https://cassandra.apache.org/_/ecosystem.html
  - Tutorial Cassandra Query Language (CQL): https://www.tutorialspoint.com/cassandra/index.htm
- Bootcamp 2022 for Apache Cassandra

  https://github.com/datastaxdevs/bootcamp-fullstack-apps-with-cassandra
- DBeaver Community: https://dbeaver.io/

Comentario final

Es fundamental tener en cuenta que la calificación de este trabajo práctico no dependerá exclusivamente del correcto funcionamiento del código CQL propuesto.

Un peso muy significativo de la nota recaerá sobre la **calidad, claridad y rigor de la documentación entregada**. Se espera que el estudiante elabore una memoria completa, bien redactada y estructurada, que vaya más allá de la simple exposición de capturas de pantalla. A la hora de corregir, se valorará especialmente:

- La **justificación técnica** y el razonamiento detrás de las decisiones de diseño adoptadas (muy especialmente en el modelado de las tablas y la elección de claves).

- El **análisis descriptivo** de las dificultades o errores encontrados durante el despliegue de la infraestructura o la ejecución de consultas, explicando cómo se diagnosticaron y resolvieron.
- La **capacidad de reflexión y síntesis** mostrada en las conclusiones, valorando el espíritu crítico y las opiniones fundamentadas sobre la tecnología estudiada.

En definitiva, se busca evaluar tanto la destreza técnica con Apache Cassandra como la capacidad de comunicar y argumentar el trabajo realizado.